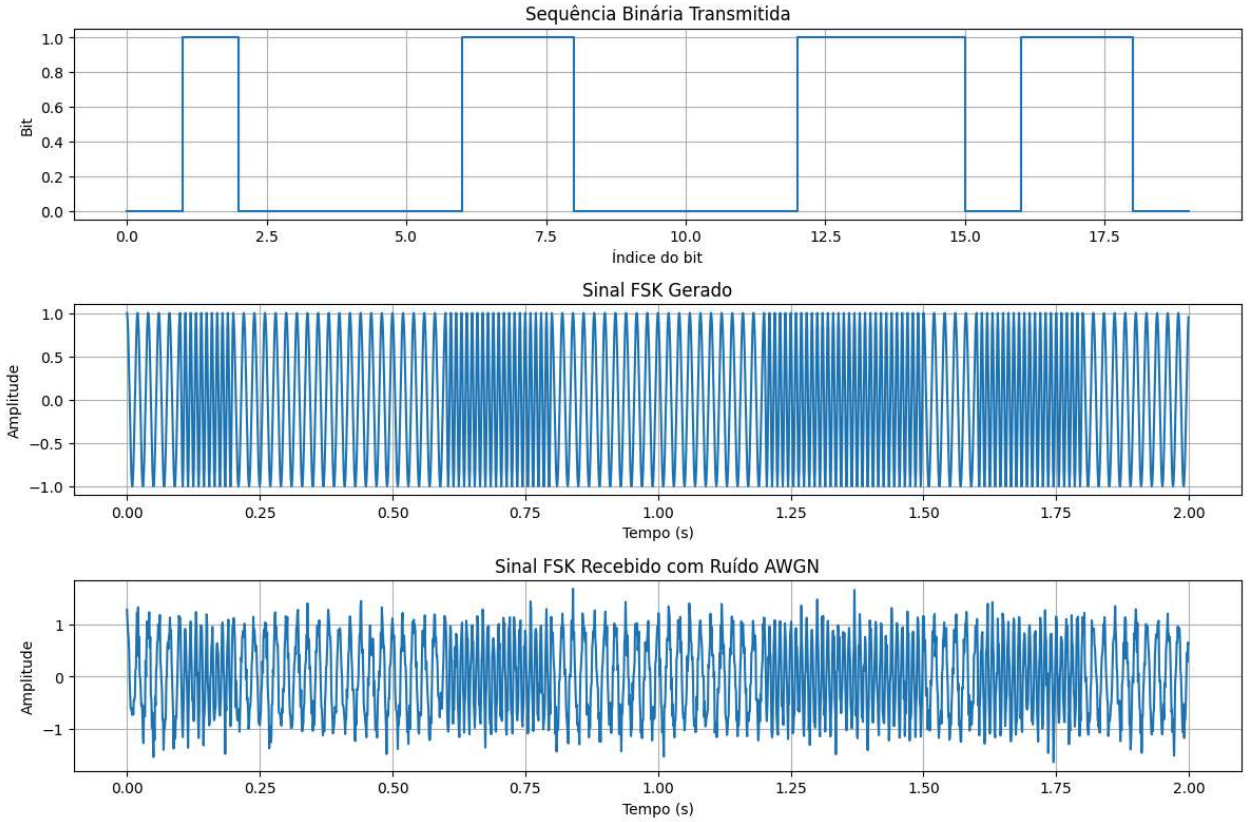
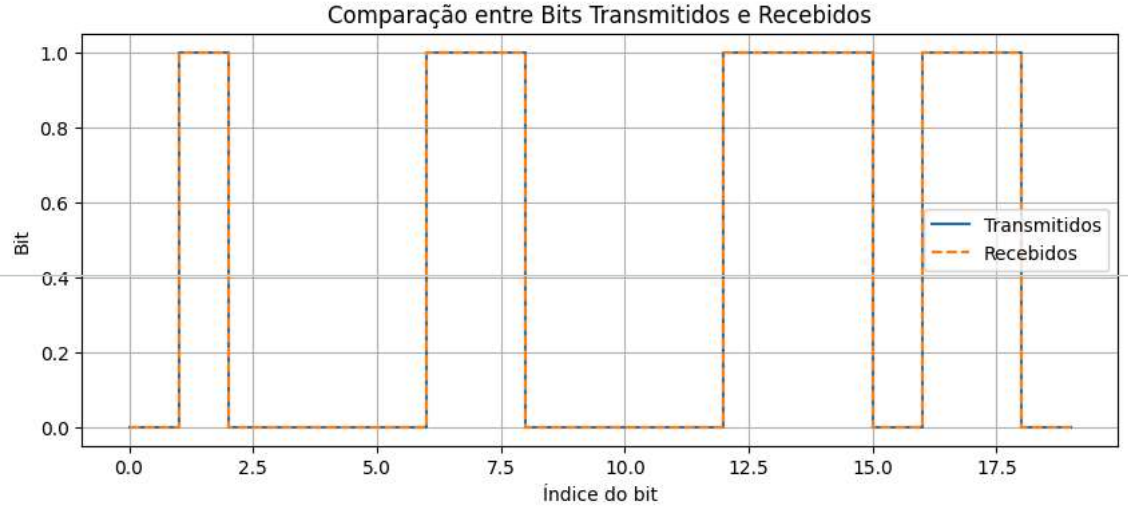
# Simulação de Gerador e Receptor de Sinais FSK

Github: janiramarques7-cpu/TP-547-Principios-de-Simulacao-em-Sistemas-de-Telecomunicacoes: Agiana 978

Nome: Agiana da Silva

```
# Simulação de Gerador e Receptor de Sinais FSK

import numpy as np
import matplotlib.pyplot as plt


# 1. Parâmetros da simulação


num_bits = 20                # Quantidade de bits transmitidos
Rb = 10                      # Taxa de bits em bits por segundo
Tb = 1 / Rb                  # Duração de cada bit
fs = 1000                    # Frequência de amostragem
A = 1                        # Amplitude do sinal FSK

f0 = 50                      # Frequência usada para representar o bit 0
f1 = 100                     # Frequência usada para representar o bit 1

SNR_dB = 10                  # Relação sinal-ruído em dB


# 2. Geração da sequência binária

bits = np.random.randint(0, 2, num_bits)

print("Bits transmitidos:")
print(bits)


# 3. Geração do sinal FSK

samples_per_bit = int(fs * Tb)
t_bit = np.arange(samples_per_bit) / fs

fsk_signal = np.array([])

for bit in bits:
    if bit == 0:
        signal_bit = A * np.cos(2 * np.pi * f0 * t_bit)
    else:
        signal_bit = A * np.cos(2 * np.pi * f1 * t_bit)

    fsk_signal = np.concatenate((fsk_signal, signal_bit))


# Vetor de tempo total
t = np.arange(len(fsk_signal)) / fs


# 4. Adição de ruído AWGN ao canal

signal_power = np.mean(fsk_signal ** 2)
SNR_linear = 10 ** (SNR_dB / 10)
noise_power = signal_power / SNR_linear

noise = np.sqrt(noise_power) * np.random.randn(len(fsk_signal))

received_signal = fsk_signal + noise


# 5. Receptor FSK por correlação

received_bits = []

carrier_0 = np.cos(2 * np.pi * f0 * t_bit)
carrier_1 = np.cos(2 * np.pi * f1 * t_bit)

for i in range(num_bits):
    start = i * samples_per_bit
    end = start + samples_per_bit

    received_bit_signal = received_signal[start:end]
```

```
    received_bit_signal = received_signal[start:end]

    correlation_0 = np.sum(received_bit_signal * carrier_0)
    correlation_1 = np.sum(received_bit_signal * carrier_1)

    if correlation_1 > correlation_0:
        received_bits.append(1)
    else:
        received_bits.append(0)

received_bits = np.array(received_bits)

print("\nBits recebidos:")
print(received_bits)


# 6. Cálculo da taxa de erro de bit

errors = np.sum(bits != received_bits)
BER = errors / num_bits

print("\nNúmero de erros:", errors)
print("BER:", BER)


# 7. Gráficos dos resultados


plt.figure(figsize=(12, 8))

plt.subplot(3, 1, 1)
plt.step(np.arange(num_bits), bits, where='post')
plt.title("Sequência Binária Transmitida")
plt.xlabel("Índice do bit")
plt.ylabel("Bit")
plt.grid(True)

plt.subplot(3, 1, 2)
plt.plot(t, fsk_signal)
plt.title("Sinal FSK Gerado")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(3, 1, 3)
plt.plot(t, received_signal)
plt.title("Sinal FSK Recebido com Ruído AWGN")
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout()
plt.show()


# 8. Comparação visual dos bits transmitidos e recebidos

plt.figure(figsize=(10, 4))

plt.step(np.arange(num_bits), bits, where='post', label="Transmitidos")
plt.step(np.arange(num_bits), received_bits, where='post', linestyle='--', label="Recebidos")

plt.title("Comparação entre Bits Transmitidos e Recebidos")
plt.xlabel("Índice do bit")
plt.ylabel("Bit")
plt.legend()
plt.grid(True)
plt.show()
```

```
Bits transmitidos:
[0 1 0 0 0 0 1 1 0 0 0 0 1 1 1 0 1 1 0 0]

Bits recebidos:
[0 1 0 0 0 0 1 1 0 0 0 0 1 1 1 0 1 1 0 0]

Número de erros: 0
BER: 0.0
```